

FUNDAMENTOS DE PYTHON

PYTHON É IGUAL CAFÉ: RESOLVE A MAIORIA DOS PROBLEMAS



Caroline Perez Gonçalves
2026

FUNDAMENTOS DE PYTHON

Caroline Perez Gonçalves

SUMÁRIO

APRESENTAÇÃO.....	1
INTRODUÇÃO	2
O QUE É PYTHON?.....	3
HISTÓRIA DO PYTHON	4
POR QUE APRENDER PYTHON?.....	6
PREPARANDO O AMBIENTE DE DESENVOLVIMENTO.....	8
NOSSO OBJETIVO	9
VARIÁVEIS	9
TIPOS DE DADOS.....	13
AGRADECIMENTO.....	18
CONTATO.....	19

APRESENTAÇÃO

Aos que não me conhecem, meu nome é Caroline. Sou formada em Análise e Desenvolvimento de Sistemas, pós-graduada em Inteligência Artificial e Machine Learning e possuo MBA em Gerenciamento de Projetos.

Além da minha experiência profissional com tecnologia, projetos e equipes multidisciplinares, também atuo como instrutora. Sempre acreditei que o conhecimento deve ser compartilhado de forma simples, leve e acessível, e foi com esse propósito que esta apostila foi desenvolvida.

O intuito deste material é mostrar que o Python está presente em muito mais situações do que imaginamos e que entender essa tecnologia não precisa ser algo complicado. Muito mais do que aprender ferramentas, o mais importante é compreender os conceitos, entender como a linguagem funciona, onde ela pode ser aplicada e como ela pode transformar as coisas.

Ao final desta apostila, espero que você tenha construído uma base sólida sobre os fundamentos dessa linguagem e se sinta mais confiante para acompanhar as próximas trilhas de estudo e explorar esse universo com mais segurança.

Vamos aos estudos! Mas, antes, sente-se, relaxe e sinta-se em casa. Quero que este material seja como uma boa conversa entre amigos, tornando o aprendizado mais leve, agradável e, principalmente, significativo.

Boa sorte e muito sucesso!

INTRODUÇÃO

Afinal, por que aprender Python?

A resposta para essa e outras perguntas tem um segredo. Muita gente acredita que programação é um assunto complicado, cheio de códigos difíceis ou exclusivo para quem trabalha com tecnologia. Mas a verdade é justamente o contrário.

Como assim, Carol?

A programação está presente em praticamente tudo o que fazemos. Ela está por trás dos aplicativos que usamos diariamente, dos sites que acessamos, dos sistemas bancários, das compras online, dos aplicativos de transporte, das redes sociais e até de muitos equipamentos que fazem parte da nossa rotina.

Quando você entende isso, percebe que aprender Python não significa apenas estudar uma linguagem de programação. Significa desenvolver uma nova forma de pensar, aprender a resolver problemas de maneira lógica e criar soluções para situações do dia a dia.

E tem mais: aprender Python não é decorar comandos. Os comandos podem até mudar de uma linguagem para outra, mas o que realmente faz diferença é compreender os conceitos, desenvolver o raciocínio lógico e entender como construir programas capazes de automatizar tarefas e facilitar o trabalho das pessoas.

Hoje o Python é utilizado em diversas áreas, como desenvolvimento de sistemas, automação, análise de dados, Inteligência Artificial, desenvolvimento Web, computação em nuvem, segurança da informação e muitas outras. Isso faz dele uma das linguagens mais versáteis e procuradas do mercado.

Claro, existem assuntos mais avançados que exigem bastante estudo e prática. Mas, como em qualquer aprendizado, tudo começa pelos fundamentos. E é exatamente isso que vamos construir ao longo desta apostila.

Para que essa introdução não fique muito extensa, vamos explorar cada tema no momento certo. E, como todo bom aprendizado, vamos começar pelo começo.

Vamos lá.

O QUE É PYTHON?

Antes de aprender Python, existe uma pergunta que precisamos responder.

O que é Python?

Muita gente acredita que Python é um programa. Outros imaginam que seja um aplicativo ou até mesmo uma ferramenta exclusiva para quem trabalha com Inteligência Artificial.

Mas não.

Python é uma **linguagem de programação**.

E o que isso significa?

Imagine que você quer ensinar uma pessoa a fazer um bolo. Você precisa explicar os ingredientes, a quantidade de cada um, a ordem correta e o tempo de forno.

Com um computador acontece algo parecido. Ele não consegue entender frases como:

"Abra o sistema e calcule o valor da venda."

Para que isso aconteça, precisamos escrever essas instruções em uma linguagem que o computador consiga interpretar.

É exatamente para isso que existe uma linguagem de programação. Ela funciona como uma ponte entre nós e o computador.

Nós escrevemos instruções seguindo determinadas regras, e o computador executa aquilo que foi solicitado. Python é uma dessas linguagens.

Mas por que justamente ela?

Porque foi criada pensando em simplicidade.

Enquanto algumas linguagens possuem uma sintaxe mais complexa, cheia de símbolos e comandos difíceis para quem está começando, o Python procura ser mais próximo da forma como pensamos. Isso faz com que a leitura do código seja muito mais simples.

É por esse motivo que Python se tornou uma das linguagens mais utilizadas no mundo, tanto por quem está aprendendo a programar quanto por profissionais com muitos anos de experiência.

E não pense que Python serve apenas para criar pequenos programas. Muito pelo contrário. Hoje ela está presente em diversas áreas da tecnologia. Alguns exemplos são:

- Desenvolvimento de sistemas;
- Desenvolvimento Web;

- Automação de tarefas;
- Ciência de Dados;
- Inteligência Artificial;
- Machine Learning;
- Análise de Dados;
- Computação em Nuvem (Cloud);
- Cibersegurança;
- Internet das Coisas (IoT);
- Desenvolvimento de APIs;
- Testes automatizados.

Ou seja... Quando você aprende Python, não está aprendendo apenas uma linguagem. Está aprendendo uma ferramenta que pode abrir portas para diversas carreiras diferentes.

Talvez você queira desenvolver sistemas.

Talvez queira trabalhar com dados.

Talvez queira criar automações para economizar horas de trabalho.

Ou quem sabe desenvolver aplicações utilizando Inteligência Artificial.

Independentemente do caminho escolhido, existe uma grande chance de o Python estar presente. É uma curiosidade interessante...

Se você estava esperando encontrar aquele famoso **"Hello World"**, pode ficar tranquilo. Ele tem sua importância histórica, mas dificilmente alguém será contratado para escrever "Olá, Mundo!" na tela.

Ao longo desta trilha, vamos utilizar exemplos muito mais próximos da realidade. Vamos criar programas que fazem sentido no dia a dia, entender como resolver problemas e aprender a pensar como um desenvolvedor.

Afinal, aprender uma linguagem não significa decorar comandos. Significa aprender a criar soluções.

HISTÓRIA DO PYTHON

Agora que já entendemos o que é Python, surgiu outra pergunta.

De onde essa linguagem veio?

Muita gente imagina que Python surgiu por causa da Inteligência Artificial. Outros

acreditam que foi criada recentemente, acompanhando o crescimento da tecnologia.

Mas a história é bem diferente.

O Python nasceu no final da década de 80 e foi lançado oficialmente em **1991**. Nem ousem dizer “faz tempo!”, porque eu sou quase desse ano, viu?!

Seu criador é um programador holandês chamado **Guido van Rossum** (sorte que isso é uma apostila e não um vídeo, pois ia ser uma pronuncia ma-ra-vi-lho-sa).

Naquela época, já existiam diversas linguagens de programação. O problema é que muitas eram difíceis de aprender, possuíam uma sintaxe complicada e exigiam bastante experiência para desenvolver até mesmo programas simples.

Guido acreditava que programar poderia ser uma experiência mais agradável.

Seu objetivo não era criar a linguagem mais rápida do mundo. Também não era criar a linguagem mais poderosa. A ideia era desenvolver uma linguagem que fosse **simples de aprender, fácil de ler e agradável de utilizar**.

E conseguiu.

Uma das frases mais conhecidas relacionadas ao Python diz:

"O código é lido muito mais vezes do que é escrito."

Pense por um instante. Imagine que você desenvolveu um sistema hoje. Daqui a seis meses, provavelmente precisará voltar naquele código para fazer alguma alteração.

Talvez outra pessoa da equipe também precise entender o que você escreveu.

Se o código estiver confuso, cheio de abreviações e difícil de interpretar, qualquer manutenção se torna muito mais complicada. Foi justamente pensando nisso que o Python foi criado.

Ele valoriza a **legibilidade**, ou seja, procura fazer com que o código seja fácil de entender, tanto para quem escreveu quanto para quem vai lê-lo no futuro. Aliás, existe uma curiosidade interessante (pelo menos eu acho).

Apesar do nome, **Python não recebeu esse nome por causa da cobra**.

Muita gente acredita nisso.

Na verdade, Guido van Rossum era fã de um grupo de humor britânico chamado **Monty Python**.

Quando decidiu criar a linguagem, escolheu "Python" como uma homenagem ao grupo. Ou seja, nome da linguagem não tem relação com o animal. A cobra acabou se tornando um símbolo da linguagem apenas mais tarde.

Com o passar dos anos, o Python começou a conquistar espaço em diferentes áreas.

Primeiro entre programadores que buscavam produtividade. Depois nas universidades.

Mais tarde, em empresas de tecnologia. E atualmente está presente em organizações do mundo inteiro.

Hoje, Python é utilizado por startups, pequenas empresas, grandes indústrias, instituições financeiras, universidades, órgãos públicos e algumas das maiores empresas de tecnologia do planeta.

Isso aconteceu porque a linguagem consegue atender necessidades muito diferentes.

Com ela é possível criar desde pequenos scripts para automatizar tarefas até sistemas completos, aplicações web, ferramentas de análise de dados, soluções de Inteligência Artificial, automações, APIs e muito mais.

Mas talvez o maior motivo do sucesso do Python seja outro.

Ele permite que o desenvolvedor concentre seus esforços na resolução do problema, e não na complexidade da linguagem.

Em outras palavras... Você gasta menos tempo tentando entender a sintaxe e mais tempo criando soluções. E isso faz toda a diferença.

Afinal, quando começamos a aprender programação, nosso objetivo não é decorar comandos. É aprender a resolver problemas utilizando a tecnologia.

POR QUE APRENDER PYTHON?

Depois de conhecer um pouco da história do Python, talvez você esteja se perguntando:

"Mas por que eu deveria aprender justamente essa linguagem?"

A resposta é simples. Porque ela consegue unir três características que poucas linguagens possuem ao mesmo tempo, que é a facilidade de aprendizado, grande quantidade de aplicações e alta demanda no mercado de trabalho.

Quando uma pessoa aprende programação pela primeira vez, ela já precisa lidar com vários conceitos novos: lógica, variáveis, condições, repetições, funções... Imagine ter que enfrentar tudo isso utilizando uma linguagem complicada.

Seria como aprender a dirigir começando por um caminhão carregado, em vez de um carro mais comum, pequeno e do dia a dia.

O Python facilita esse processo. Sua sintaxe é organizada, intuitiva e bastante próxima da forma como pensamos.

Isso significa que você passa menos tempo tentando entender comandos e mais tempo aprendendo programação de verdade.

Mas não pense que Python é apenas uma linguagem para iniciantes.

Esse é um dos maiores mitos sobre ela. O fato de ser fácil de aprender não significa que seja limitada. Muito pelo contrário.

Hoje ela é utilizada em empresas do mundo inteiro para desenvolver soluções de diferentes tamanhos. Com Python podemos criar:

- Sistemas empresariais;
- Aplicações Web;
- APIs;
- Automações;
- Programas para análise de dados;
- Inteligência Artificial;
- Machine Learning;
- Jogos;
- Testes automatizados;
- Ferramentas para segurança da informação;
- Scripts para administração de servidores.

É uma linguagem extremamente versátil.

Outro ponto importante é a enorme comunidade que existe ao redor do Python. Milhões de pessoas utilizam essa linguagem diariamente.

Isso significa que existem milhares de tutoriais, fóruns, vídeos, documentações, bibliotecas e exemplos disponíveis na internet.

Em outras palavras...

Se surgir uma dúvida durante seus estudos, existe uma grande chance de alguém já ter passado pela mesma situação.

Além disso, aprender Python também pode abrir portas no mercado de trabalho.

Empresas procuram profissionais que saibam resolver problemas utilizando tecnologia. E Python aparece frequentemente entre as linguagens mais solicitadas em vagas relacionadas a desenvolvimento, análise de dados, automação, Inteligência Artificial, computação em nuvem e diversas outras áreas.

Mas quero deixar uma observação importante.

Aprender Python, por si só, não garante um emprego.

Assim como comprar uma caixa de ferramentas não transforma alguém em um excelente mecânico. A linguagem é apenas uma ferramenta.

O que realmente fará diferença será a sua capacidade de criar soluções, entender problemas e aplicar o conhecimento adquirido. E é exatamente isso que vamos construir ao longo desta trilha. Entenda: o objetivo não é apenas aprender comandos.

O objetivo é aprender a pensar como um programador de verdade.

PREPARANDO O AMBIENTE DE DESENVOLVIMENTO

Muito bem. Agora chegou a hora de colocar a mão na massa. Mas antes de escrever nosso primeiro programa, precisamos preparar o computador.

Pense da seguinte forma (ainda utilizando o exemplo de uma caixa de ferramenta). Antes de construir qualquer coisa, é preciso organizar o espaço de trabalho, separar as ferramentas e deixá-las prontas para uso.

Na programação acontece exatamente a mesma coisa.

Antes de escrever código, precisamos instalar algumas ferramentas. E a boa notícia é que esse processo é bem simples. Basicamente, vamos utilizar duas delas.

Python

A primeira ferramenta é o próprio Python. Ele será responsável por interpretar o código que escreveremos.

Em outras palavras, sempre que criarmos um programa, será o Python quem fará a leitura das instruções e executará aquilo que pedimos. Sem ele, o computador não saberia o que fazer com nossos arquivos.

Por isso, o primeiro passo é instalar a linguagem em seu computador. Durante a instalação, você encontrará uma opção chamada:

"Add Python to PATH"

Se essa opção aparecer, marque-a antes de continuar.

Ela permite que o Python seja encontrado facilmente pelo sistema operacional, evitando alguns problemas comuns para quem está começando.

Pode parecer um detalhe, mas esquecer essa opção costuma gerar bastante dor de cabeça depois.

Visual Studio Code

Depois de instalar o Python, precisamos de um lugar confortável para escrever nossos programas. Poderíamos utilizar o bloco de notas? Sim, ele funciona. Mas seria como construir uma casa usando apenas uma chave de fenda.

Existem ferramentas muito melhores para isso.

Uma das mais utilizadas atualmente é o **Visual Studio Code**, também conhecido como **VS Code**.

Ele é um editor de código gratuito, leve e extremamente popular entre

desenvolvedores.

Além de permitir escrever programas, ele oferece diversos recursos que facilitam o desenvolvimento, como destaque de sintaxe, organização do código, sugestões automáticas e integração com diversas extensões.

Ao instalar o VS Code, também será interessante instalar a extensão oficial do Python. Ela adiciona vários recursos que tornam o desenvolvimento muito mais confortável.

Não se preocupe se esses nomes parecerem novos. Com o tempo, eles se tornarão seus amigos (ou inimigos).

NOSSO OBJETIVO

Se você já instalou o Python e o VS Code, parabéns. Seu ambiente de desenvolvimento está praticamente pronto. E sabe o que isso significa?

Significa que, a partir do próximo assunto, vamos começar a escrever nossos primeiros códigos. Mas pode ficar tranquilo(a). Prometo manter a promessa feita lá no início da apostila...

Aqui não vamos perder tempo criando exemplos sem utilidade apenas porque todo curso faz isso.

Vamos aprender Python resolvendo situações reais, entendendo o motivo de cada comando e construindo uma base sólida para tudo o que veremos nas próximas apostilas.

E como eu sempre pego uma frase para repetir freneticamente, quero dizer que aprender programação não é decorar código. É aprender a criar soluções!

VARIÁVEIS

Até agora, já entendemos o que é Python, conhecemos um pouco da sua história e preparamos nosso ambiente de desenvolvimento. Agora chegou a hora de começar a programar.

Mas antes existe uma pergunta muito importante.

Como um programa consegue "lembrar" de uma informação?

Imagine que você foi contratado para desenvolver um sistema para uma loja. Esse sistema precisa guardar várias informações.

- Nome do cliente;
- Valor da compra;
- Quantidade de produtos;
- Endereço;

- Telefone;
- E-mail.

Se o computador não conseguisse armazenar essas informações, todo o trabalho seria perdido a cada nova operação. É exatamente para resolver esse problema que existem as **variáveis**.

Uma variável é um espaço utilizado para armazenar uma informação que poderá ser utilizada durante a execução do programa.

Pense em uma variável como uma caixa. Na parte de fora da caixa existe uma etiqueta com um nome. Dentro dela está guardada uma informação. Imagine a seguinte situação.

Na etiqueta está escrito:

nome_cliente

E dentro da caixa está armazenado:

"Marcos Oliveira"

Sempre que precisarmos dessa informação, basta procurar pela etiqueta.

Na programação acontece exatamente a mesma coisa.

Criamos um nome para identificar uma informação que queremos guardar. Veja alguns exemplos.

```
nome_cliente = "Marcos Oliveira"
```

```
idade = 34
```

```
salario = 4850.75
```

```
cliente_ativo = True
```

Observe que sempre existe um nome à esquerda e um valor à direita.

O sinal de igualdade (=) não significa exatamente "é igual". Na programação, ele representa uma **atribuição**. Em outras palavras, estamos dizendo ao Python:

"Guarde este valor dentro desta variável."

Depois disso, podemos utilizar essa informação sempre que precisarmos. Veja um exemplo.

```
print(nome_cliente)
```

O resultado será:

Marcos Oliveira

Perceba uma coisa interessante. Nós não escrevemos novamente o nome do cliente.

Pedimos ao Python para mostrar o conteúdo armazenado naquela variável.

Isso torna os programas muito mais organizados. Imagine um sistema de uma clínica médica e o paciente altera o número de telefone.

Se essa informação estivesse escrita manualmente em vários lugares diferentes do sistema, seria necessário atualizar tudo.

Mas, como ela está armazenada em uma variável, basta alterar aquele único valor. Todo o restante do programa utilizará automaticamente a informação atualizada.

É por isso que praticamente qualquer software utiliza variáveis.

Sem elas, seria praticamente impossível desenvolver sistemas maiores.

Escolhendo bons nomes

Uma das maiores diferenças entre um código profissional e um código feito apenas para estudar está na escolha dos nomes das variáveis. Veja este exemplo.

```
a = 250
```

```
b = "Notebook"
```

```
c = True
```

Consegue dizer rapidamente o que cada variável representa? Provavelmente não.

Agora observe este outro exemplo.

```
preco_produto = 250
```

```
nome_produto = "Notebook"
```

```
produto_disponivel = True
```

Muito melhor, não é? Mesmo alguém que nunca viu esse código consegue entender o que está acontecendo.

Por isso, sempre escolha nomes que façam sentido. Alguns exemplos.

- nome_cliente;
- valor_total;
- quantidade_produtos;
- email_usuario;
- pedido_finalizado;
- estoque_disponivel.

Evite utilizar nomes como a, x, abc, teste, valor1 ou variavel2.

Pode parecer algo pequeno, mas um código bem escrito facilita muito a manutenção e o trabalho em equipe.

Lembre-se de uma frase que você já leu por aqui: "O código é lido muito mais vezes do que é escrito".

Variáveis podem mudar

Existe um motivo para elas receberem esse nome. Elas podem variar.

Imagine o estoque de uma loja. Hoje existem 50 unidades de um produto. Após uma venda, passam a existir 49. A informação mudou. Veja um exemplo.

```
estoque = 50
```

```
estoque = 49
```

Perceba que utilizamos a mesma variável. Apenas alteramos o valor armazenado. Isso acontece o tempo todo em sistemas reais.

- O saldo de uma conta bancária muda;
- A temperatura muda;
- O preço de um produto muda;
- A quantidade em estoque muda;
- O status de um pedido muda;
- O número de acessos de um site muda.

As variáveis existem justamente para armazenar essas mudanças.

Uma curiosidade

Talvez você esteja pensando:

"Mas onde essas informações ficam guardadas?"

Quando executamos um programa, o Python utiliza a memória do computador para armazenar temporariamente essas informações.

Não precisamos nos preocupar com os detalhes técnicos agora.

O importante, neste momento, é entender que uma variável funciona como um local onde podemos guardar informações para utilizá-las sempre que necessário durante a execução do programa.

Mais adiante, aprenderemos como salvar essas informações em arquivos e bancos de dados para que elas continuem existindo mesmo depois que o programa for

encerrado.

TIPOS DE DADOS

No assunto anterior aprendemos que as variáveis servem para armazenar informações. Mas agora surgiu uma nova pergunta (pelo menos eu me perguntei isso).

Como o Python sabe qual tipo de informação está sendo armazenada?

Vamos imaginar uma situação bem simples. Você foi contratado para desenvolver um sistema de cadastro de funcionários. Para cada pessoa será necessário armazenar algumas informações.

- Nome completo;
- Idade;
- Salário;
- Funcionário ativo.

À primeira vista, tudo parece apenas "informação". Mas repare em um detalhe.

Cada uma delas possui uma característica diferente.

O nome é um texto.

A idade é um número inteiro.

O salário possui casas decimais.

Já a informação sobre o funcionário estar ativo ou não possui apenas duas possibilidades.

Sim ou não. Verdadeiro ou falso.

É exatamente por isso que existem os **tipos de dados**.

Eles informam ao Python qual é a natureza da informação que estamos armazenando. Também pode parecer algo pequeno, mas imagine um sistema que tentasse somar dois nomes. Ou dividir um endereço por cinco. Ou multiplicar um e-mail por dez. Não faria sentido.

Por isso, antes de realizar qualquer operação, o Python precisa saber com que tipo de dado está trabalhando.

Os quatro tipos mais utilizados são:

- String (str);
- Inteiro (int);
- Decimal (float);

- Booleano (bool).

Vamos conhecer cada um deles.

String (str)

Uma **string** representa qualquer informação textual.

Sempre que estivermos trabalhando com palavras, frases ou qualquer sequência de caracteres, estaremos utilizando esse tipo de dado.

Alguns exemplos são:

- Nome de uma pessoa;
- Cidade;
- Endereço;
- CPF;
- E-mail;

Observe o código abaixo.

```
nome = "Caroline"
```

```
cidade = "São Paulo"
```

```
email = "cliente@email.com"
```

Perceba que todos os textos aparecem entre aspas. As aspas informam ao Python que aquele conteúdo deve ser tratado como um texto.

Mesmo quando um texto contém apenas números, ele continua sendo uma string. Por exemplo.

```
cpf = "12345678900"
```

Embora existam apenas números, um CPF não será utilizado para realizar cálculos matemáticos. Por isso, ele deve ser armazenado como texto.

O mesmo acontece com CEP, telefone, placa de veículo e diversos outros dados.

Inteiro (int)

O tipo **int** representa números inteiros. Ou seja, números que não possuem casas decimais. Alguns exemplos são:

- Idade;
- Quantidade de produtos;
- Número de funcionários;
- Total de clientes;
- Ano de fabricação.

Veja alguns exemplos.

idade = 32

quantidade = 18

ano = 2026

clientes = 850

Sempre que estivermos contando alguma coisa, normalmente utilizaremos números inteiros.

Decimal (float)

Nem todos os números são inteiros. Em muitas situações precisamos trabalhar com valores que possuem casas decimais. Por exemplo:

- Preço de um produto;
- Salário;
- Temperatura;
- Altura;
- Peso.

Veja alguns exemplos.

preco = 1899.90

salario = 4850.75

temperatura = 27.5

altura = 1.78

Esses valores pertencem ao tipo **float**.

O nome pode parecer estranho no começo. Mas você vai se acostumar rapidamente com ele.

Sempre que houver casas decimais, normalmente estaremos utilizando esse tipo de dado.

Booleano (bool)

Agora vamos conhecer um tipo de dado um pouco diferente.

Imagine um aplicativo bancário. Depois que uma transferência é realizada, o sistema precisa responder apenas uma pergunta. A transferência foi concluída?

As respostas possíveis são apenas duas: sim ou não.

O mesmo acontece em diversas outras situações.

- O pagamento foi aprovado?
- O usuário está logado?
- O pedido foi entregue?
- O estoque está disponível?

Percebe? Sempre existem apenas duas respostas possíveis.

Para esses casos utilizamos o tipo **booleano**, conhecido como **bool**. Veja alguns exemplos.

```
pedido_pago = True
```

```
usuario_logado = False
```

```
produto_disponivel = True
```

```
email_confirmado = False
```

Nesse tipo de dado utilizamos apenas dois valores: True ou False.

Em português: verdadeiro ou falso.

Os booleanos aparecem o tempo todo em sistemas reais. Sempre que uma decisão depender de uma resposta do tipo "sim ou não", provavelmente estaremos trabalhando com um valor booleano.

Como o Python identifica o tipo?

Uma curiosidade interessante é que, na maioria das vezes, não precisamos informar o tipo da variável. O próprio Python consegue identificar automaticamente. Veja este exemplo.

```
nome = "Caroline"
```

```
idade = 32
```

salario = 4850.75

ativo = True

Sem que precisemos informar nada, o Python entende que o nome é uma string, a idade é um inteiro, o salario é um número decimal e o ativo é um booleano.

Essa capacidade de identificar automaticamente o tipo de dado torna a linguagem muito mais simples para quem está começando.

Por que isso é importante?

Talvez você esteja pensando:

"Tudo isso parece muito simples."

E realmente é. Mas praticamente todos os programas do mundo utilizam esses quatro tipos de dados.

Quando você faz uma compra pela internet.

Quando agenda uma consulta médica.

Quando faz um PIX.

Quando pede comida por um aplicativo.

Quando reserva um hotel.

Quando cria uma conta em uma rede social.

Em todos esses sistemas existem milhares de variáveis armazenando textos, números inteiros, valores decimais e informações verdadeiras ou falsas.

Esses pequenos conceitos são a base da programação. Sem eles, seria impossível construir aplicações maiores.

AGRADECIMENTO

Se você chegou até aqui, quero te dizer: obrigada.

De verdade.

Finalizar um material assim não é só sobre aprender conteúdo. É sobre dedicação, tempo e vontade de evoluir. E só de você estar aqui, já mostra muito sobre você.

Essa apostila foi criada com um propósito muito claro: mostrar que conhecimento não precisa ser complicado. Que assuntos que parecem difíceis podem, sim, ser entendidos quando explicados da forma certa.

E mais do que isso: que conhecimento deve ser para todos.

Por isso, este material também foi pensado para ser acessível. Ele conta com versão em Braille, porque inclusão não é um diferencial — é o mínimo.

Todo mundo merece aprender. Todo mundo merece ter acesso.

Se, de alguma forma, esse conteúdo te ajudou a entender melhor, a se organizar ou até a enxergar as coisas de um jeito diferente, então ele já cumpriu o seu papel.

Obrigada por fazer parte disso.

E lembre-se: você não precisa complicar para fazer dar certo.

CONTATO

Se você quiser continuar aprendendo, trocar ideias ou acompanhar mais conteúdos sobre projetos, tecnologia e organização no dia a dia, você pode me encontrar por aqui:

Instagram: @CAROL_G_INTECH

LinkedIn: Caroline Gonçalves

Site: <https://cpgconsulting.com.br/>

E-mail: cpgtechconsulting@gmail.com

Fique à vontade para se conectar ou mandar uma mensagem. Vou adorar continuar essa troca com você.